Big Data Processing and Performance Benchmarking

A Comparative Study of Pandas, Chunking, and Dask

Practical Work – TP02

Prepared by: Noureddine Ben Attous

14 October 2026

Course topic:	Processing large datasets (5 GB+)
Dataset:	Amazon ratings CSV, approximately 9.06 GB
Execution platform:	Google Colab
Methods evaluated:	Standard Pandas, Pandas Chunking, Dask DataFrame

A practical evaluation of execution time, memory consumption, and result consistency.

Contents

Abstract **3**

1 Introduction **3**

2 Objectives **3**

3 Environment and Dataset **4**
3.1 Execution Environment 4
3.2 Dataset Description 4

4 Methodology **4**
4.1 Standard Pandas: Full-File Loading 4
4.2 Pandas Chunking 4
4.3 Dask DataFrame 5
4.4 Performance Measurement 5

5 Experimental Results **5**
5.1 Standard Pandas 5
5.2 Pandas Chunking 5
5.3 Dask DataFrame 6

6 Performance Comparison **6**
6.1 Execution Time Analysis 6
6.2 Memory Analysis 6

7 Visual Analysis **7**

8 Validation and Correctness **8**

9 Discussion **8**

10 Conclusion **9**

References **9**

Abstract

Processing large CSV files can create significant memory and performance challenges when the entire dataset is loaded into memory at once. This practical work compares three approaches for processing an Amazon ratings dataset of approximately 9.06 GB: standard Pandas, Pandas Chunking, and Dask DataFrame. The experiments were conducted in Google Colab, and the methods were evaluated using execution time, sampled process memory, and the consistency of their calculated statistics.

The standard Pandas experiment was intentionally stopped by a configured memory-safety limit before the dataset could be fully loaded. Pandas Chunking and Dask completed the analysis and returned matching values for the record count, valid-rating count, rating sum, and average rating. Chunking completed in 288.31 seconds with a sampled peak process memory of 0.74 GB, while Dask completed in 450.39 seconds with a sampled peak main-process memory of 1.10 GB. For this workload and environment, chunking offered the best measured balance between execution time and memory usage.

Keywords: Big Data, Pandas, Chunking, Dask, memory profiling, lazy evaluation, performance benchmarking.

1 Introduction

The growth of data-intensive applications has made efficient processing of large datasets an important practical skill. CSV files containing millions of records may require substantial memory when loaded into a single in-memory data structure. If the memory required by an operation exceeds the available resources, the program may slow down, be terminated, or raise a memory-related exception.

This practical work investigates three approaches to processing a large Amazon ratings dataset:

- **Standard Pandas:** attempts to load the complete CSV file into one DataFrame.
- **Pandas Chunking:** reads and processes the dataset in smaller batches.
- **Dask DataFrame:** represents the dataset as partitions and evaluates computations when requested.

The purpose is not to declare one tool universally superior, but to compare their behavior under the specific workload and execution environment used in this experiment.

2 Objectives

The main objectives are to:

- Understand the memory limitations of loading large files with standard `pandas.read_csv()`.
- Process a large CSV file incrementally using chunks of 100,000 rows.
- Explain Dask's partition-based processing and lazy evaluation.
- Measure execution time and sampled process memory.
- Compare the statistics returned by the completed approaches.
- Discuss the trade-offs between memory consumption, execution time, and computational overhead.

3 Environment and Dataset

3.1 Execution Environment

The experiments were performed in Google Colab. The environment measurements recorded before the tests were approximately:

- Total RAM: 12.67 GB.
- Available RAM at the beginning: 11.45 GB.
- Free disk space at the beginning: 87.21 GB.
- Main libraries: Pandas, Dask, psutil, Matplotlib, and PyArrow.

These values describe the environment at the time of the experiment and may vary between Colab sessions.

3.2 Dataset Description

The dataset is an Amazon ratings CSV file with an approximate size of 9.06 GB, exceeding the 5 GB threshold specified in the practical-work instructions. It contains 233,055,327 records and the following four columns:

- `item_id`: identifier of the rated item.
- `user_id`: identifier of the user who submitted the rating.
- `rating`: numerical rating value.
- `timestamp`: timestamp associated with the rating.

The analysis focuses on counting records and valid ratings, calculating the sum of rating values, and computing the average rating.

4 Methodology

4.1 Standard Pandas: Full-File Loading

The first experiment attempted to load the entire CSV file into one Pandas DataFrame using `read_csv()`. This approach is straightforward, but the DataFrame and intermediate parsing operations may require considerably more memory than the file's on-disk size.

To protect the runtime, the experiment was executed in a separate process with a configured RAM safety threshold based on 50% of the available memory before the test. The process was stopped after approximately 74.53 seconds, when sampled child-process memory reached 5.40 GB.

This is a controlled stop caused by the safety mechanism. It is **not evidence of a confirmed `MemoryError`** and should not be reported as an observed out-of-memory crash.

4.2 Pandas Chunking

The second experiment used Pandas with `chunksize=100000`. Rather than loading all records at once, the program reads a batch, calculates its partial statistics, updates running totals, and then proceeds to the next batch.

The main metrics are:

- Total number of records processed.
- Number of valid rating values.
- Sum of valid ratings.

- Global average rating.

The global mean is computed from the accumulated sum and count:

$$\bar{x} = \frac{\sum_{i=1}^{N} x_i}{N},$$

where x_i is a valid rating and N is the total number of valid ratings. Computing the mean from the global sum and count avoids incorrectly averaging the means of individual chunks when chunk sizes differ.

Chunking reduces memory pressure because only a portion of the dataset needs to be held in memory at a time. Its processing is generally sequential in this implementation, so it may take longer than a well-parallelized alternative for suitable workloads.

4.3 Dask DataFrame

The third experiment used Dask DataFrame to read the CSV file in partitions, with a configured block size of 64 MB. Dask builds a task graph describing the operations to perform. This graph is generally evaluated only when a result is requested, for example through `compute()` or `dask.compute()`.

This behavior is known as *lazy evaluation*. It allows Dask to plan and schedule work across partitions rather than immediately performing every operation when the expression is created. Depending on the scheduler and environment, tasks may run concurrently. However, partition management, scheduling, and data conversion also introduce overhead.

The Dask experiment calculated the same statistics as the chunking experiment so their outputs could be compared.

4.4 Performance Measurement

Execution time was recorded in seconds. Memory figures represent sampled process-level observations rather than a complete measurement of total system memory. In particular, the Dask figure is for the main process and may not include the full memory used by every worker. The measurements are therefore useful for comparing the recorded runs, but they should not be interpreted as perfectly equivalent measurements of total memory consumption.

5 Experimental Results

5.1 Standard Pandas

The standard Pandas process reached the configured safety threshold after 74.53 seconds. Its peak sampled child-process memory was 5.40 GB. The experiment was intentionally stopped to protect the runtime.

Consequently, the standard Pandas result is classified as *Stopped by safety limit*. Since the full computation did not finish, its duration is not a valid full-processing time and cannot be directly compared with the successful completion times of the other methods.

5.2 Pandas Chunking

Pandas Chunking completed the entire computation. The recorded results were:

- Total records: 233,055,327.
- Valid ratings: 233,055,327.
- Rating sum: 986,334,473.
- Average rating: 4.2322.

- Execution time: 288.31 seconds.
- Peak sampled process RAM: 0.74 GB.

These results show that incremental processing handled the dataset while keeping the sampled process memory relatively low.

5.3 Dask DataFrame

Dask also completed the computation successfully. Its recorded results were:

- Total records: 233,055,327.
- Valid ratings: 233,055,327.
- Rating sum: 986,334,473.
- Average rating: 4.2322.
- Execution time: 450.39 seconds.
- Peak sampled main-process RAM: 1.10 GB.

Dask returned the same statistics as Pandas Chunking. In this experiment, however, it took longer to complete the task.

6 Performance Comparison

Table 1 summarizes the measurements collected during the three experiments.

Method	Time (s)	RAM (GB)	Status
Standard Pandas	74.53	5.40	Stopped by safety limit; incomplete
Pandas Chunking	288.31	0.74	Completed successfully
Dask DataFrame	450.39	1.10	Completed successfully

Table 1: Measured execution time and sampled process memory.

6.1 Execution Time Analysis

Pandas Chunking completed in 288.31 seconds, compared with 450.39 seconds for Dask. Based on these measurements, chunking took approximately 36% less time than Dask for this particular workload:

$$\frac{450.39 - 288.31}{450.39} \times 100 \approx 36.0\%.$$

This result does not mean that Pandas Chunking is always faster than Dask. Performance depends on data types, storage speed, partition sizes, available CPU resources, scheduler configuration, and the operations being performed.

The standard Pandas run lasted 74.53 seconds, but it was stopped before completion. Its duration must therefore not be interpreted as evidence that it was the fastest method.

6.2 Memory Analysis

The sampled peak process memory was 0.74 GB for Pandas Chunking and 1.10 GB for the Dask main process. The standard Pandas process reached 5.40 GB before the safety stop.

The measurements suggest that chunking was memory-efficient in this run. Nevertheless, the values are not a perfect like-for-like comparison: the Pandas figure refers to the sampled child process, while the Dask figure refers to the main process and may exclude worker memory. A rigorous comparison of total memory would need to monitor all relevant processes or measure system-level memory consistently.

7 Visual Analysis

The experiment's two generated charts can be included below. Save the images in the same directory as this LaTeX source file and use the exact filenames shown here:

- `execution_time.png`
- `memory_usage.png`

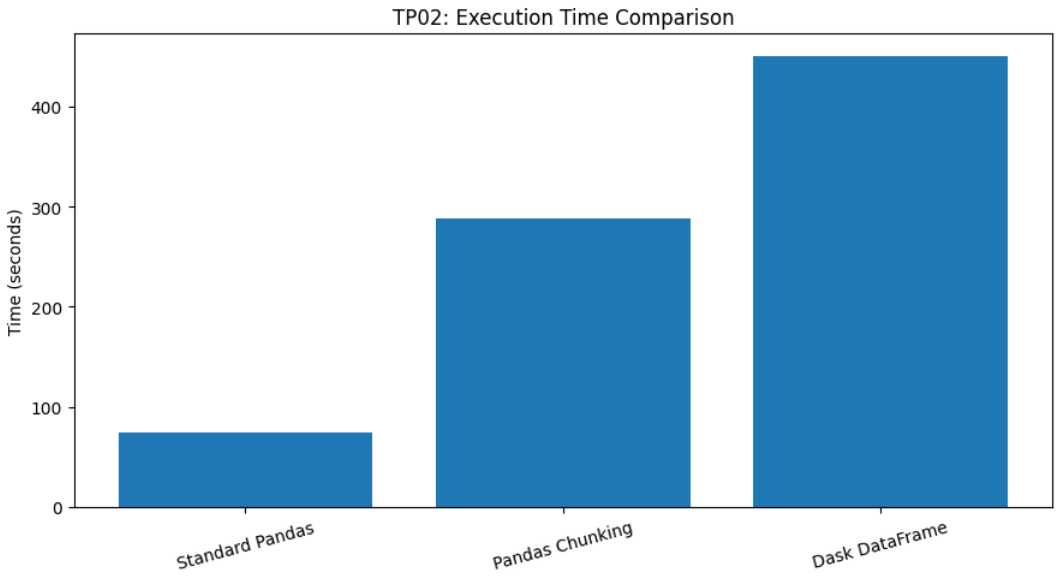


Figure 1: Execution time comparison. The standard Pandas measurement is incomplete because the process was stopped by the safety limit.

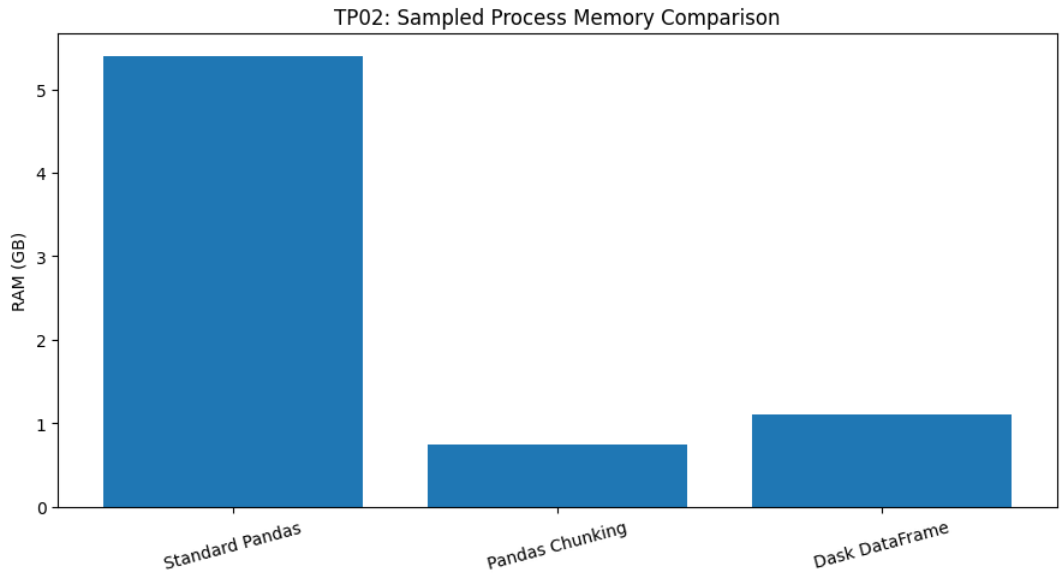


Figure 2: Sampled process memory comparison. The process scope differs between the recorded measurements.

The charts should be interpreted together with Table 1. In particular, the standard Pandas run did not complete, and its recorded time is not a valid full-task execution time.

8 Validation and Correctness

The completed Pandas Chunking and Dask runs were compared using four checks:

- Both methods processed 233,055,327 records.
- Both methods counted 233,055,327 valid ratings.
- Both methods calculated a rating sum of 986,334,473.
- Both methods reported an average rating of 4.2322 to four decimal places.

All reported validation assertions passed. This agreement supports the consistency of the two completed implementations for the statistics tested. It does not independently establish the correctness of every field in the dataset or of computations not included in the validation.

9 Discussion

The experiment demonstrates the practical trade-off between memory use, execution time, and computation overhead when processing a large CSV file.

Standard Pandas offers a simple workflow and can be effective when the complete dataset fits comfortably in memory. In this experiment, the safety mechanism stopped the full-file loading attempt before completion, so no successful processing time was obtained.

Pandas Chunking achieved the lowest recorded memory value and completed the task faster than Dask in the tested environment. It is a practical option when calculations can be expressed as incremental aggregations and the dataset is too large to load all at once.

Dask DataFrame successfully processed the dataset using partitions and lazy evaluation. It took longer in this experiment, possibly because the workload consisted of relatively simple aggregations for which task scheduling and partition management added overhead. Dask can be more beneficial when a workload can take advantage of parallel execution, multiple cores, or distributed resources.

The benchmark is specific to the dataset, Google Colab session, code, and measurement method used. Different hardware, file formats, data types, chunk sizes, partition sizes, and schedulers could change the outcome.

10 Conclusion

This practical work compared standard Pandas, Pandas Chunking, and Dask DataFrame on an Amazon ratings CSV file of approximately 9.06 GB containing 233,055,327 records.

The standard Pandas process was stopped by a configured RAM safety limit after 74.53 seconds, at a sampled memory usage of 5.40 GB. Because the run did not finish, it must not be described as a confirmed `MemoryError` or used as a successful execution-time benchmark.

Pandas Chunking and Dask both completed the computation and returned matching statistics. Chunking took 288.31 seconds and recorded 0.74 GB of sampled process RAM. Dask took 450.39 seconds and recorded 1.10 GB of sampled main-process RAM. Under the conditions of this experiment, Pandas Chunking provided the best measured balance of completion time and memory usage.

Overall, chunking is an effective technique for processing large CSV files under memory constraints, while Dask remains a useful framework for partitioned, parallel, and distributed workloads. The results also emphasize the importance of reporting measurement scope and distinguishing a controlled safety stop from a genuine out-of-memory failure.

References

1. The pandas development team. *pandas documentation: IO tools (text, CSV, HDF5, ...).* Available at: https://pandas.pydata.org/docs/user_guide/io.html
2. Dask documentation. *Dask DataFrame.* Available at: https://docs.dask.org/en/stable/dataframe.html
3. Dask documentation. *Lazy evaluation and computation.* Available at: https://docs.dask.org/en/stable/
4. psutil documentation. *Process and system utilities.* Available at: https://psutil.readthedocs.io/